

# טיפ #1: ממוצע משוקלל

ד"ר יבגני קולונסקי וד"ר יוליה פריזאנט

2025

# ממוצע של N מדידות

מדידות

$x_1, x_2 \dots x_N$

$$\mu = \frac{x_1 + \dots x_N}{N}$$

אבל! רק אם:

(1) מדידות בלתי תלויות

(2) כל המדידה שייכת לאותה ההתפלגות (נורמלית)

$$\sigma^2 = \frac{(x_1 - \mu)^2 + \dots (x_N - \mu)^2}{N - 1}$$

$$\sigma_\mu = \frac{\sigma}{\sqrt{N}}$$

# ממוצע משוקלל

$x_A$  מדידה שייכת להתפלגות A       $x_B$  מדידה שייכת להתפלגות B

לדוגמה: שתי מדידות קרובות בטווח השגיאה שלהן

$$P = P_A * P_B$$

צריך למקסם את הפונקציה

$$P \sim \exp \left[ -\frac{(x_A - \mu)^2}{\sigma_A^2} \right] \exp \left[ -\frac{(x_B - \mu)^2}{\sigma_B^2} \right]$$

$$\frac{x_A - \mu}{\sigma_A^2} + \frac{x_B - \mu}{\sigma_B^2} = 0$$

$$\mu = \frac{\frac{1}{\sigma_A^2} x_A + \frac{1}{\sigma_B^2} x_B}{\frac{1}{\sigma_A^2} + \frac{1}{\sigma_B^2}}$$

$$\sigma^2 = \frac{1}{\frac{1}{\sigma_A^2} + \frac{1}{\sigma_B^2}}$$

# קבל לוחות

| שיטת מדידה גיאומטרית          | שיטת מדידת פריקה              |
|-------------------------------|-------------------------------|
| $C = \frac{A \epsilon_0}{d}$  | $C = \frac{Q}{V}$             |
| $C_1 = 430 \pm 20 \text{ pF}$ | $C_2 = 445 \pm 10 \text{ pF}$ |

ממוצע או ממוצע משוקלל?

# קוד קצר נחמד

$$C_1 = 430 \pm 20 \text{ pF}, \quad C_2 = 445 \pm 10 \text{ pF}$$

```
import numpy as np
# capacitance measured by geometrical method
C1, sigma1 = 430, 20
# capacitance measured with oscilloscope data
C2, sigma2 = 445, 10
# weights
w1, w2 = 1/sigma1**2, 1/sigma2**2
# weighted average
C = (C1 * w1 + C2 * w2) / (w1 + w2)
# uncertainty of weighted average
sigma = 1/np.sqrt(w1 + w2)
print(f'Weighted average: {C:.0f} ± {sigma:.0f} pF')
```

Weighted average: 442 ± 9 pF

$$C = 442 \pm 9 \text{ pF}$$

# טיפ #2: מבחן $\chi^2$

יבגני קולונסקי

2025

# חי בריבוע

## Chi - square

$$\chi^2 = \sum_{i=1}^n \left( \frac{\text{ערך צפוי במודל} - \text{מדידה}}{\text{סטיית תקן}} \right)^2$$

# חי בריבוע מצומצם

## Reduced chi - square

$$\tilde{\chi}_o^2 = \frac{\chi^2}{\text{מספר דרגות חופש}}$$

$$\tilde{\chi}_o^2 \gg 1$$



$$\tilde{\chi}_o^2 < 1$$





# הגדרת "דרגות חופש"

$$y = a x^2$$

מדידות: 10  
פרמטרי התאמה : 1  
דרגות חופש: 9

$$y = a x^2 + b$$

מדידות: 10  
פרמטרי התאמה : 2  
דרגות חופש: 8

$$y = a x^2 + b x + c$$

מדידות: 10  
פרמטרי התאמה: 3  
דרגות חופש: 7

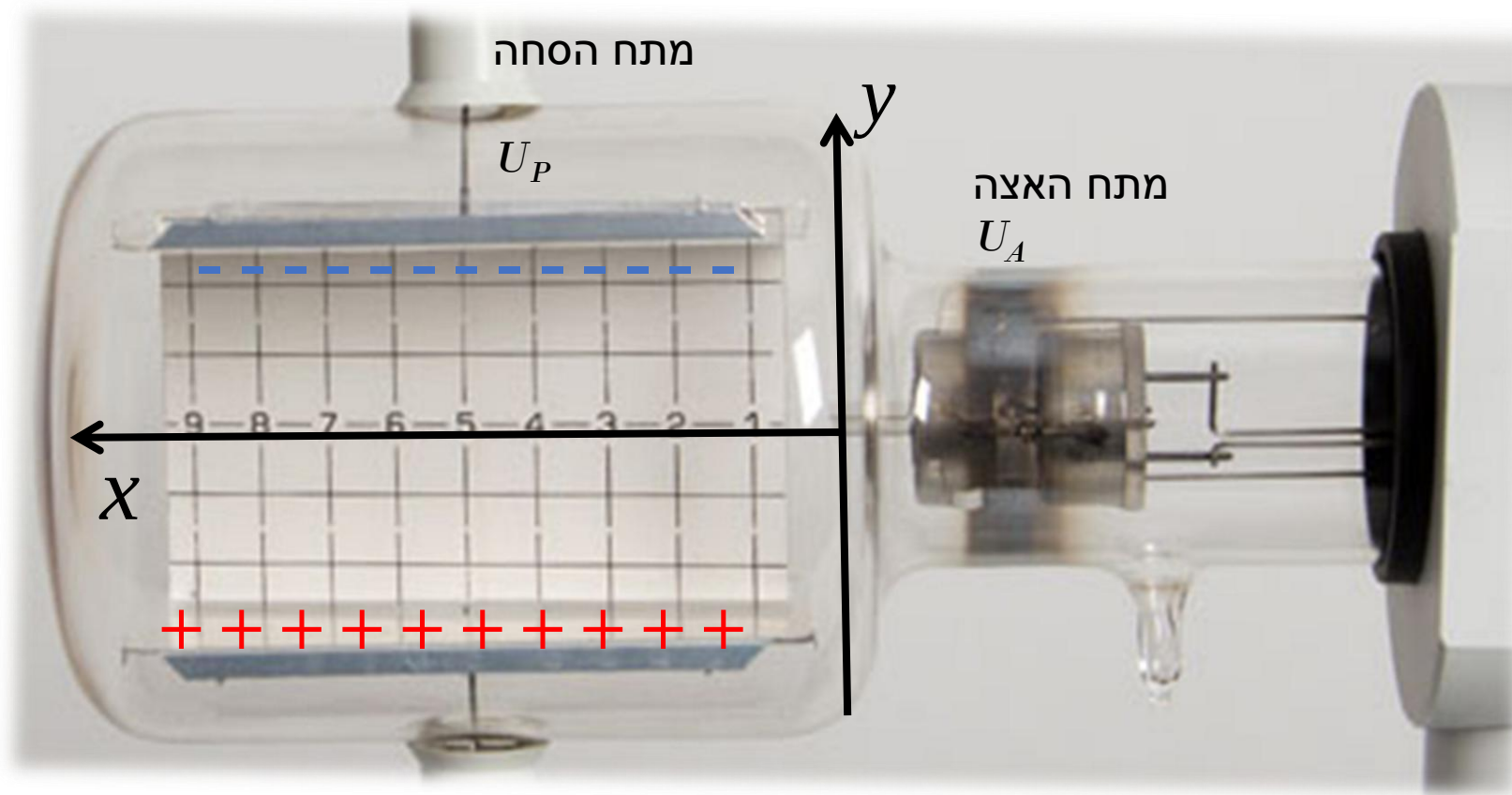
מספר דרגות חופש = מספר מדידות – מספר פרמטרי התאמה

# אלקטרון בשדה חשמלי

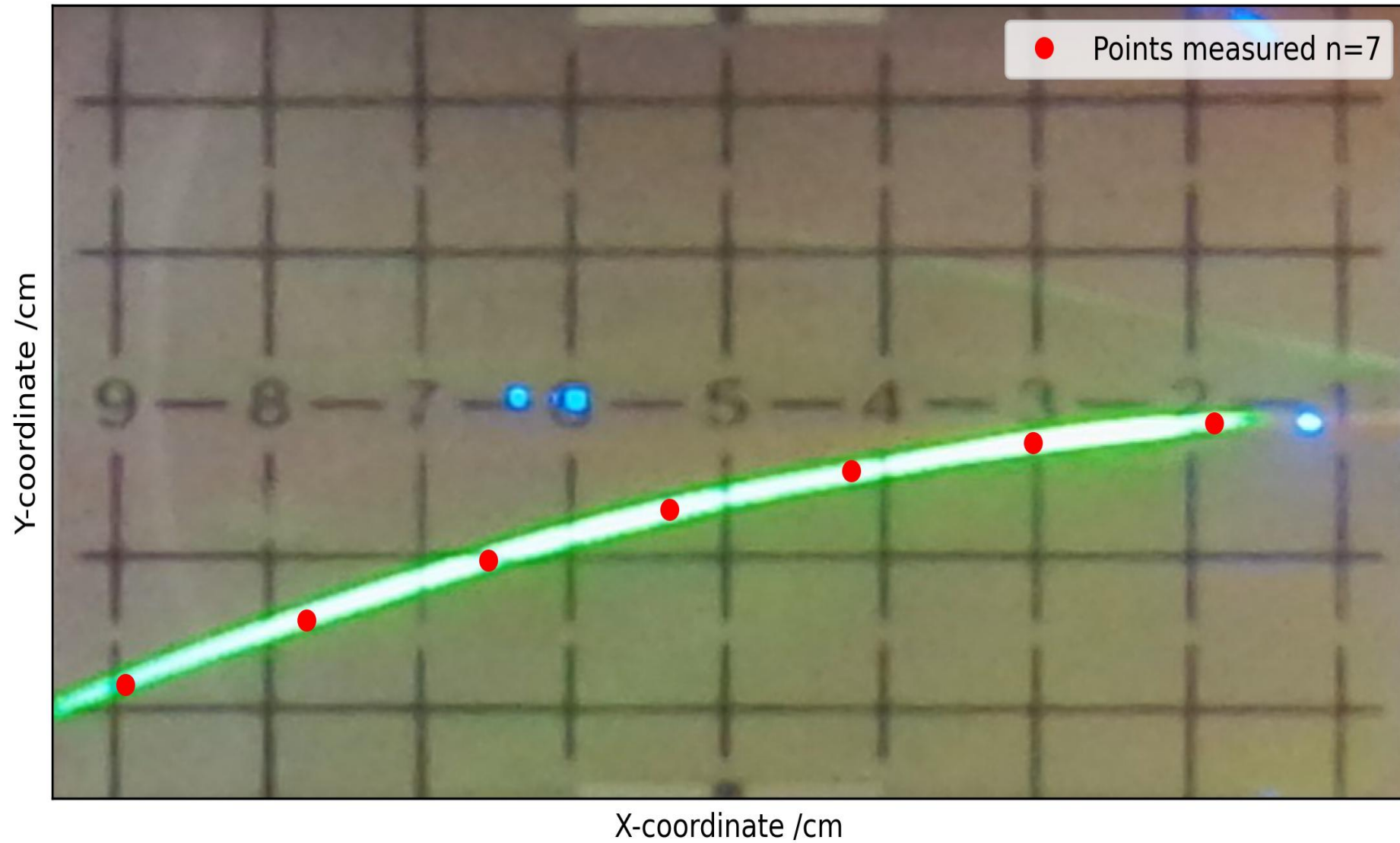
כאשר שדה חשמלי אחיד  
מסלול של אלקטרון מתואר  
על ידי:

$$y = \frac{U_P k_1}{4dU_A} x^2$$

מרחק בין לוחות –  $d$



# מסלול של אלקטרון



# קוד קצר

```
# Measured coordinates, cm
points = '''
9.02  7.81  6.60  5.40  4.19  2.98  1.78
-1.85 -1.42 -1.03 -0.70 -0.44 -0.26 -0.13'''
x, y = np.genfromtxt(points.splitlines())

# measurements uncertainty estimated, mm
sigma = 0.5e-1

# parabolic model with one parameter
def parabola(x, a):
    return a*x**2

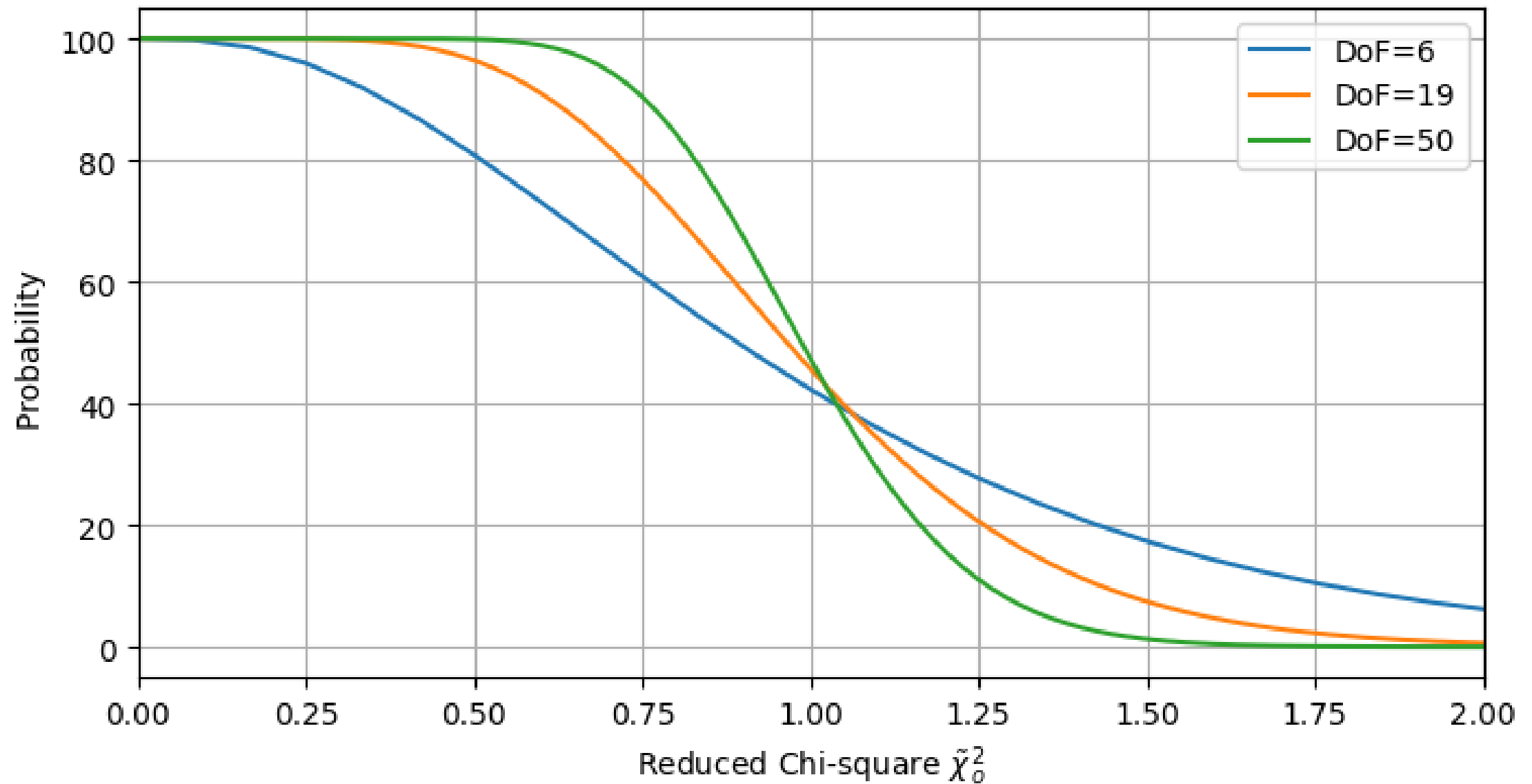
# fitting the model
parameters, _ = curve_fit(parabola, x, y)
y_expected = parabola(x, *parameters)

chi2_observed = np.sum((y - y_expected)**2 / sigma**2)
DoF = x.size - parameters.size # degrees of freedom
R2 = r2_score(y, y_expected)

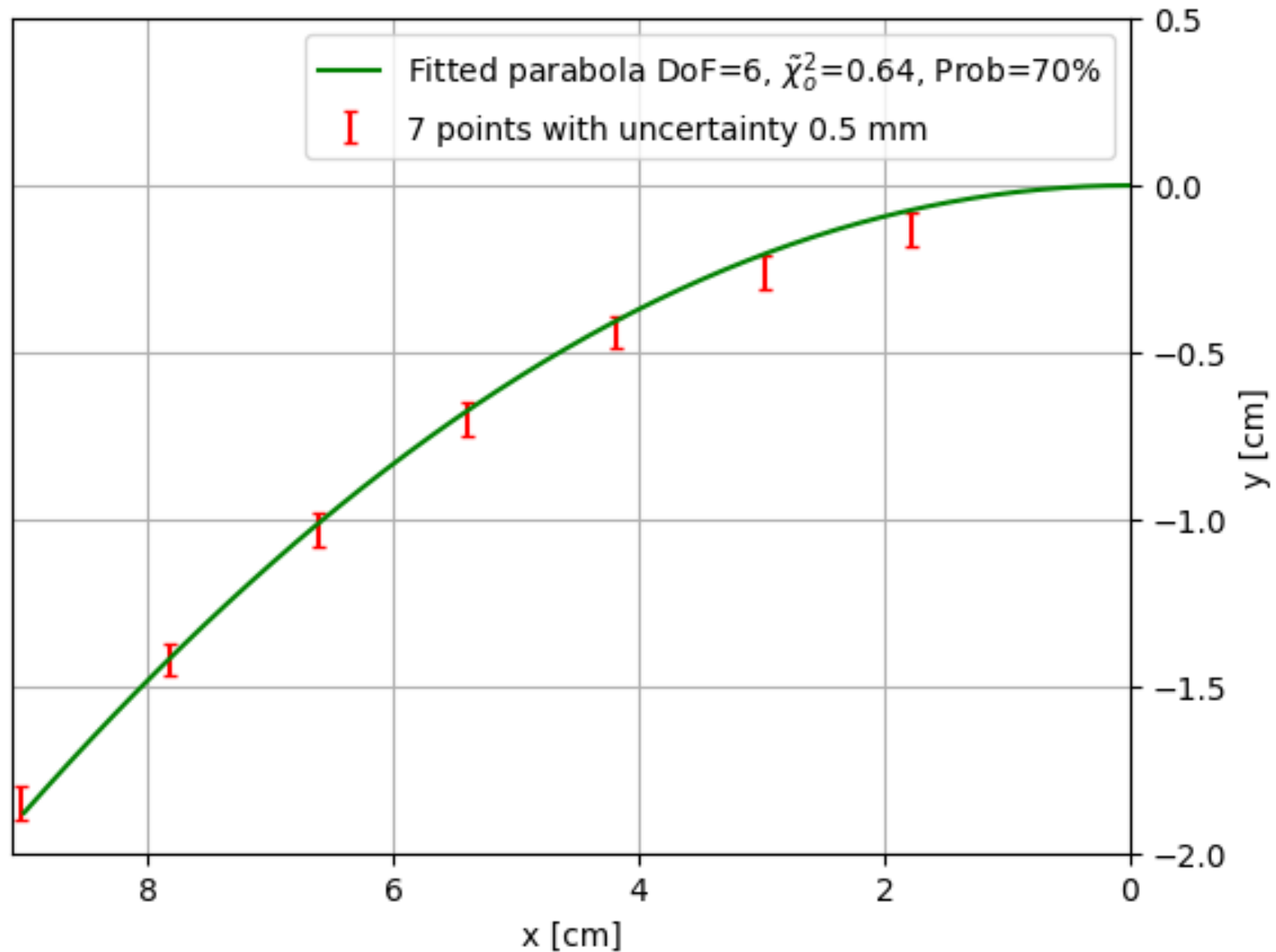
print(f'Measurements: {x.size}, Parameters: {parameters.size}, DoF: {DoF}')
print(f'Reduced Chi-squared observed {chi2_observed/DoF:.2f}')
print(f'R2 score: {r2_score(y, y_expected):.4f}')
```

```
Measurements: 7, Parameters: 1, DoF: 6
Reduced Chi-squared observed 0.64
R2 score: 0.9960
```

# התפלגות חי בריבוע

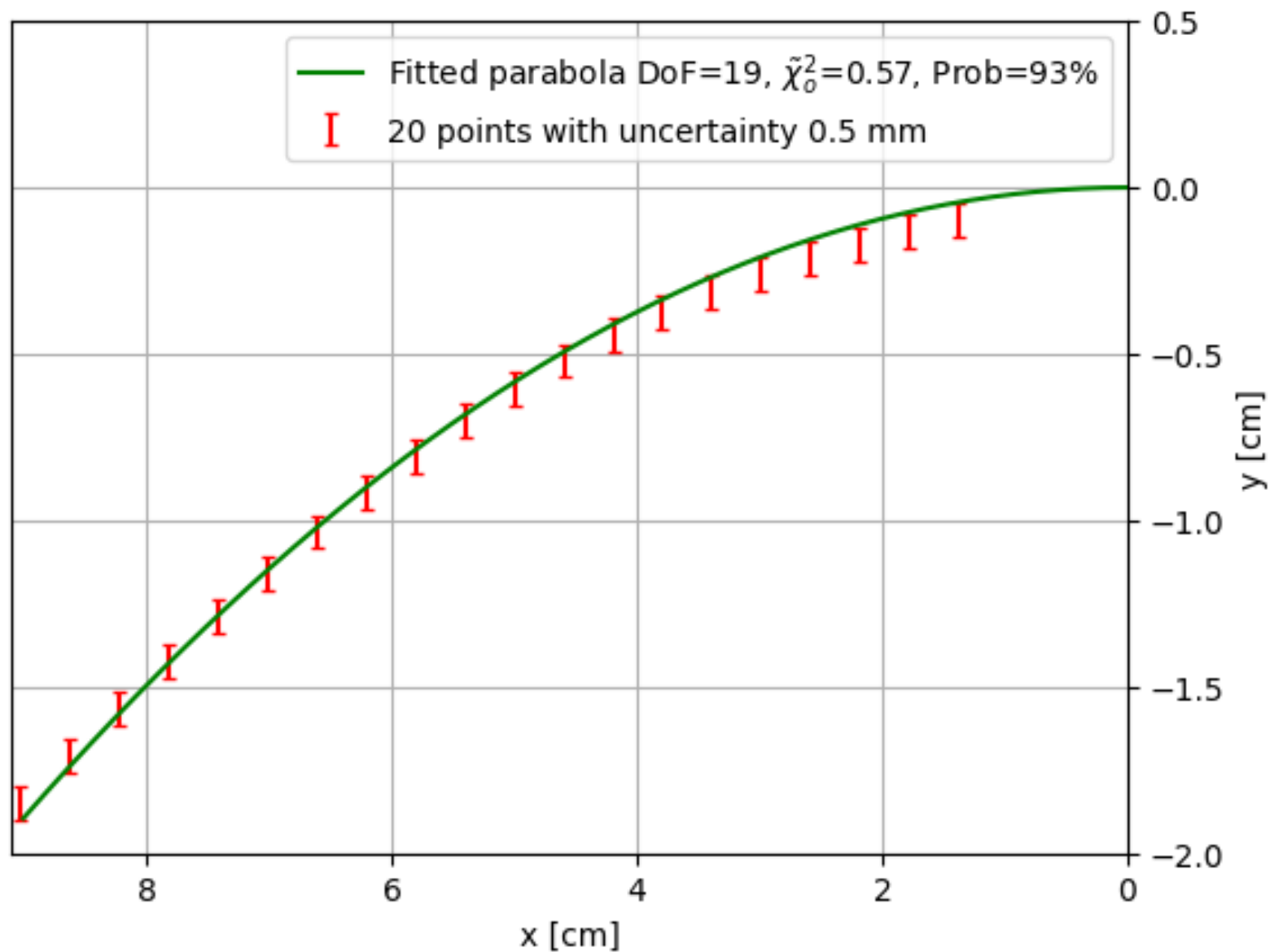


# מידת התאמה בצורה של גרף



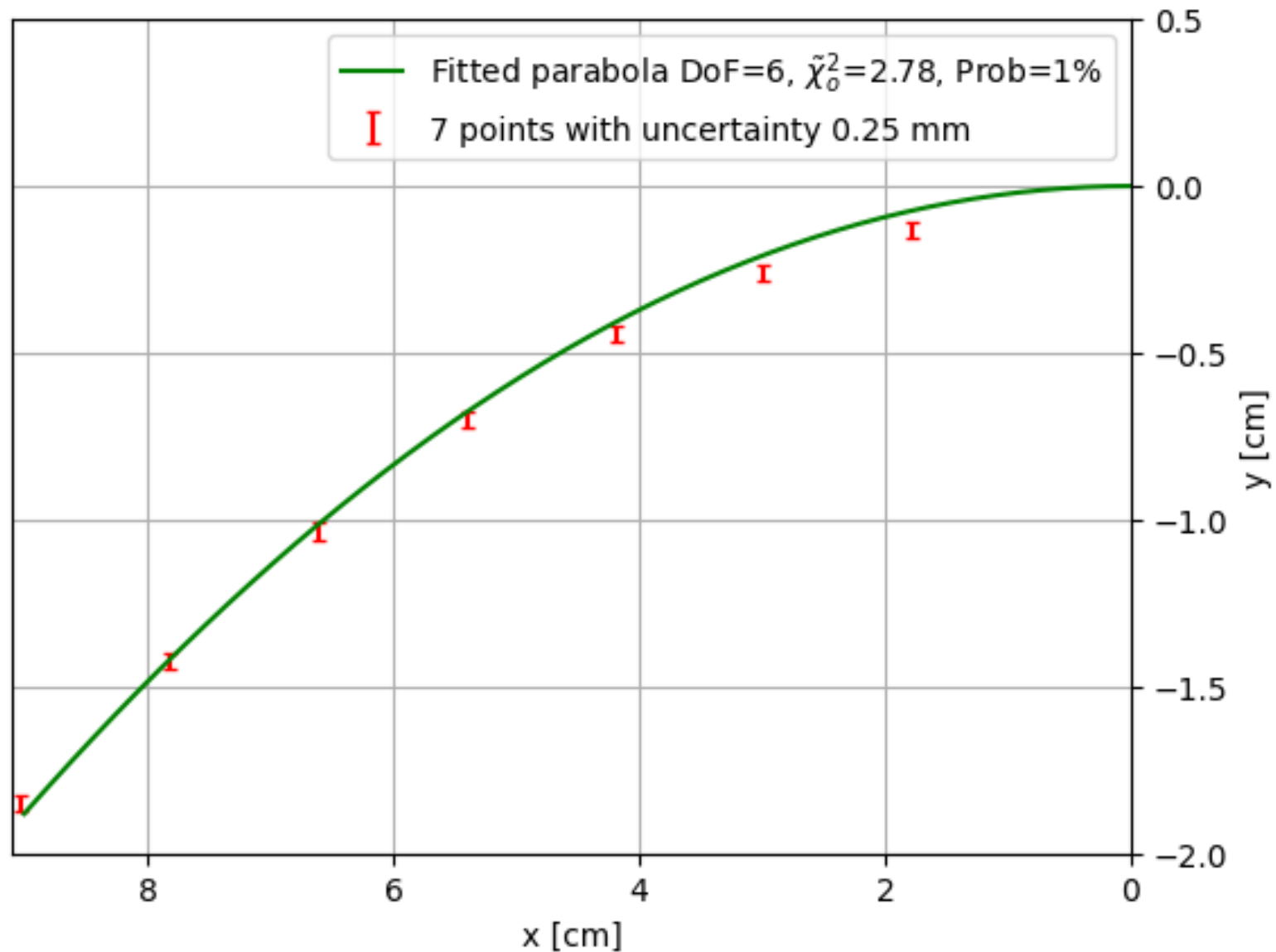
$$R^2 = 0.9960$$

# לקחנו יותר מדידות מהתמונה



$$R^2 = 0.9954$$

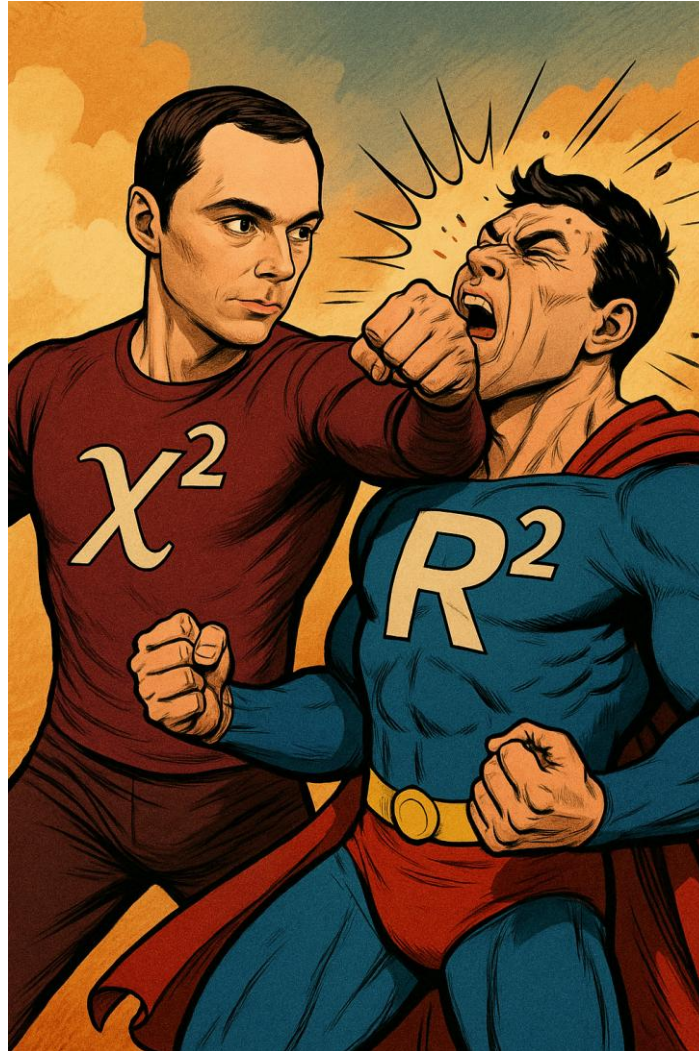
# נניח שמדדנו עם דיוק יותר גבוה



$$R^2 = 0.9960$$

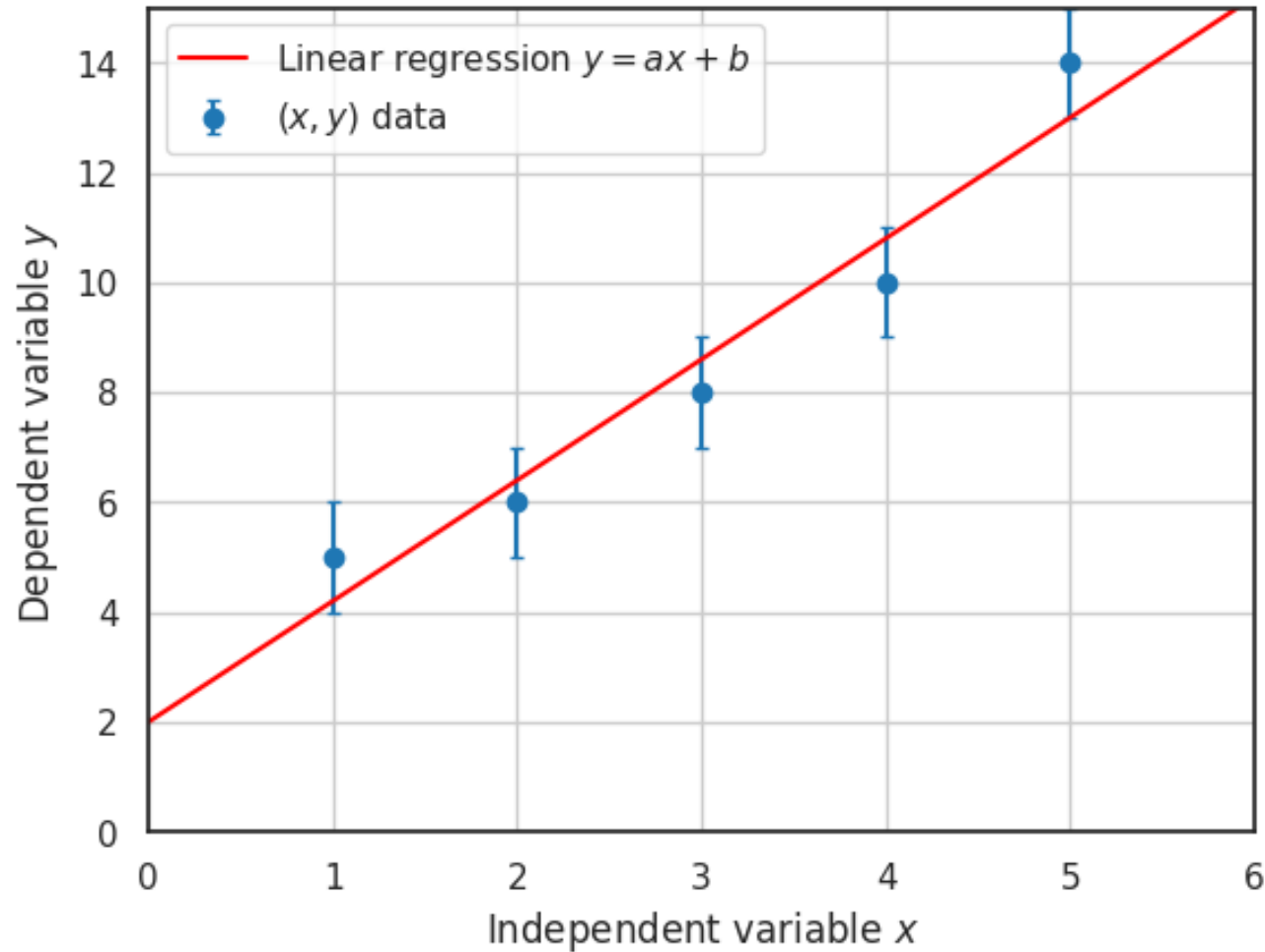


$\chi^2$  vs  $R^2$



# טיפ #3: רגרסיה לינארית: גילוי נאות

# הנחות של רגרסיה



$$y = ax + b$$

$$\sigma_x \ll \sigma_y$$

$$\sigma_y = \text{const}$$

# יישום רגרסיה בפייתון

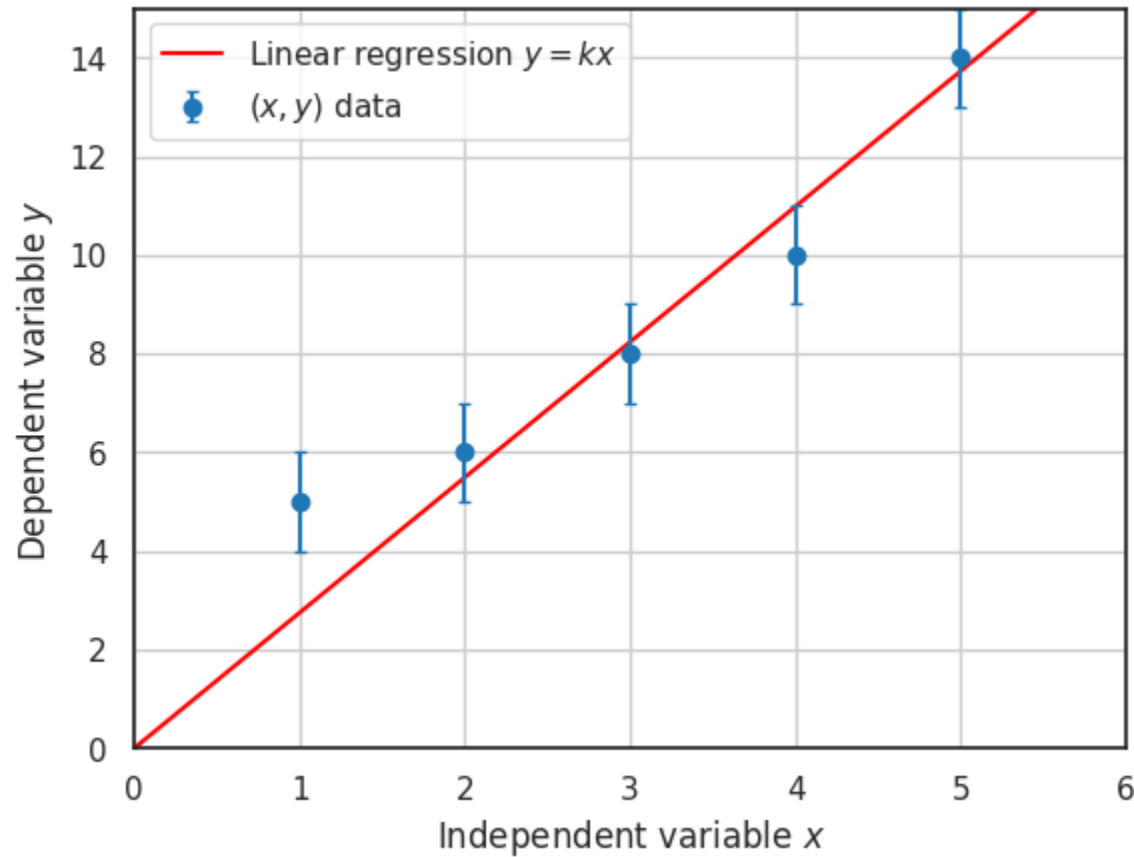
```
import numpy as np
from scipy.stats import linregress

x = np.array([1, 2, 3, 4, 5]) # Independent variable
y = np.array([5, 6, 8, 10, 14]) # Dependent variable

fit = linregress(x, y)
slope = fit.slope
intercept = fit.intercept
slope_err = fit.stderr
intercept_err = fit.intercept_stderr
print(f'Linear fit parameters:')
print(f'slope      : {slope:.1f} ± {slope_err:.1f}')
print(f'intercept: {intercept:.1f} ± {intercept_err:.1f}')
```

```
Linear fit parameters:
slope      : 2.2 ± 0.3
intercept: 2.0 ± 1.0
```

# איך לעשות רגרסיה לפונקציה ללא איבר חופשי?



$$y = k x$$

$$k = \frac{y}{x}$$

$$\frac{\sigma_k}{k} = \frac{\sigma_y}{y}$$

# יישום ממוצע משוקלל בחישוב של שיפוע

```
x = np.array([1, 2, 3, 4, 5]) # Independent variable
y = np.array([5, 6, 8, 10, 14]) # Dependent variable
y_error = 1

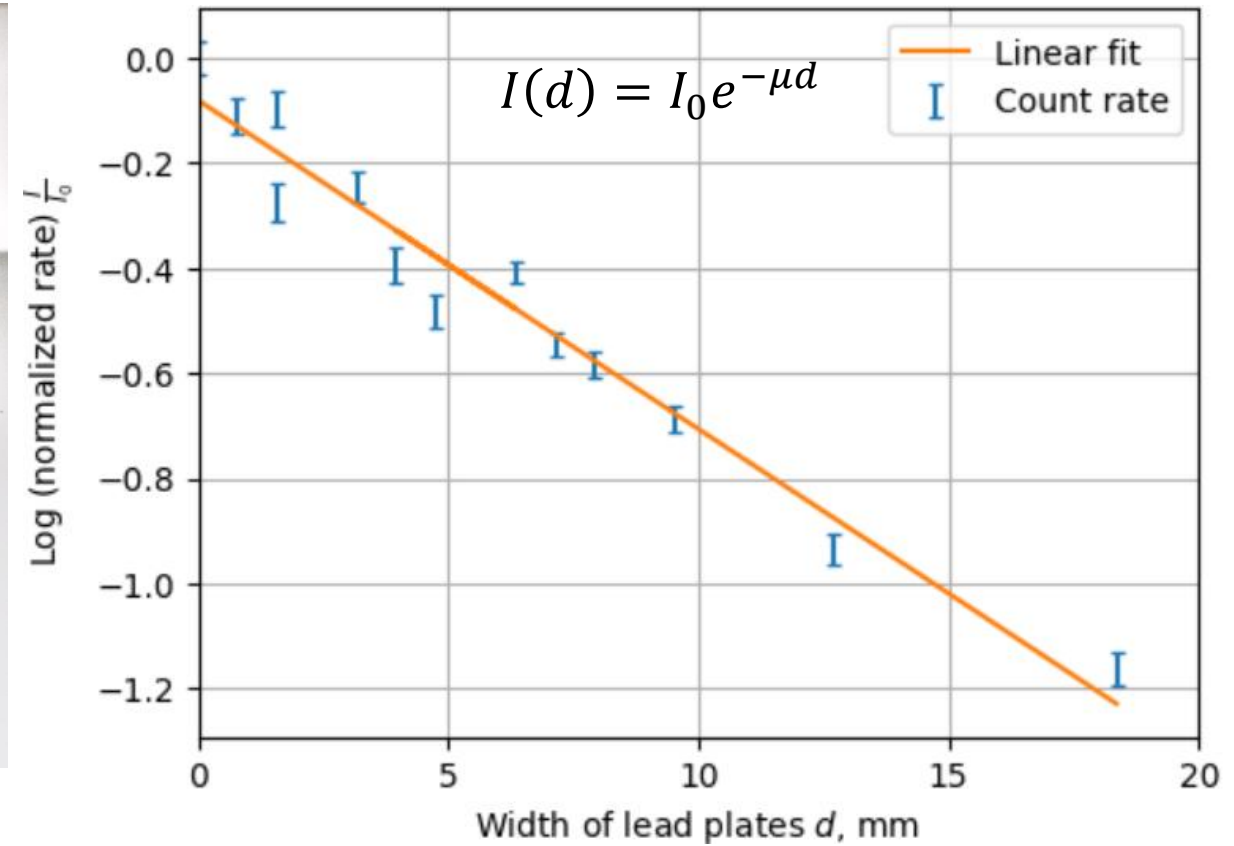
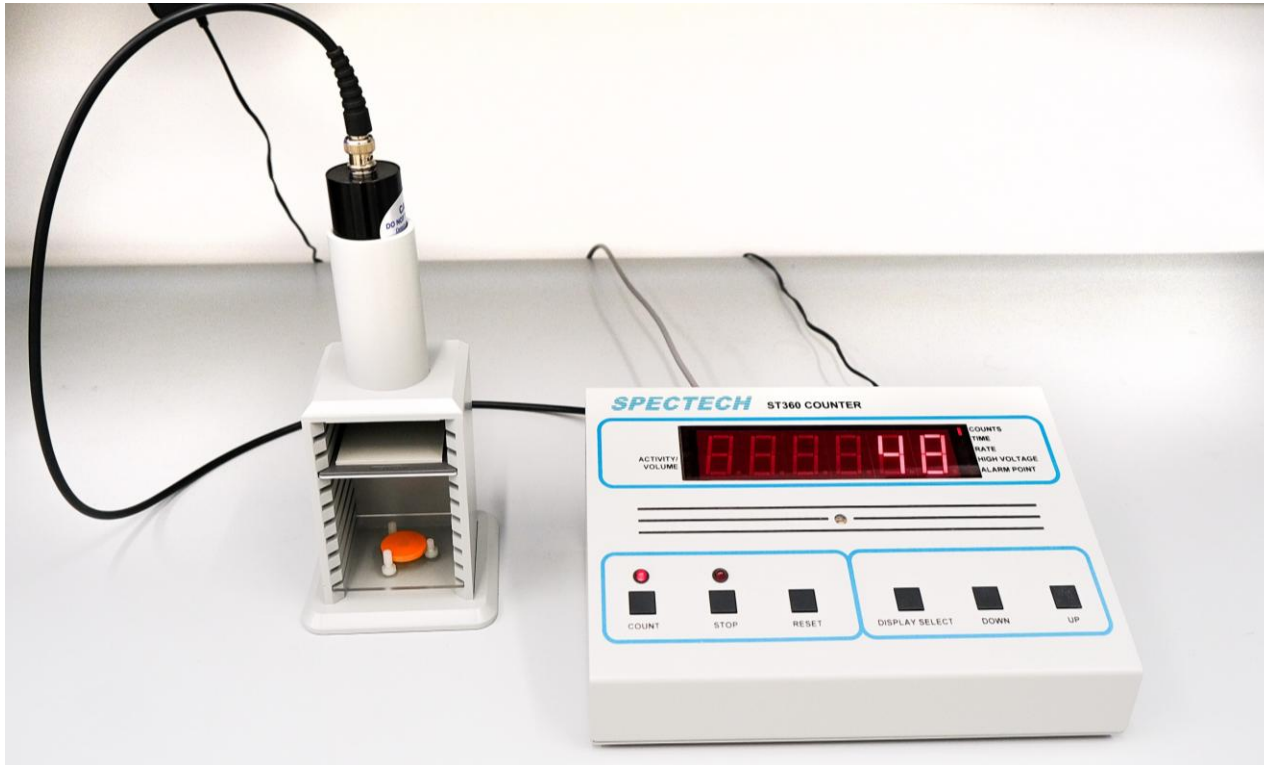
def weighted_average(values, errors):
    weights = 1 / errors**2
    average = np.sum(weights * values) / np.sum(weights)
    error_of_average = np.sqrt(1 / np.sum(weights))
    return average, error_of_average

k, k_error = weighted_average(y/x, y_error/x)
print(f'Linear fit without intercept:')
print(f'Slope      : {k:.1f} ± {k_error:.1f}')
```

Linear fit without intercept:

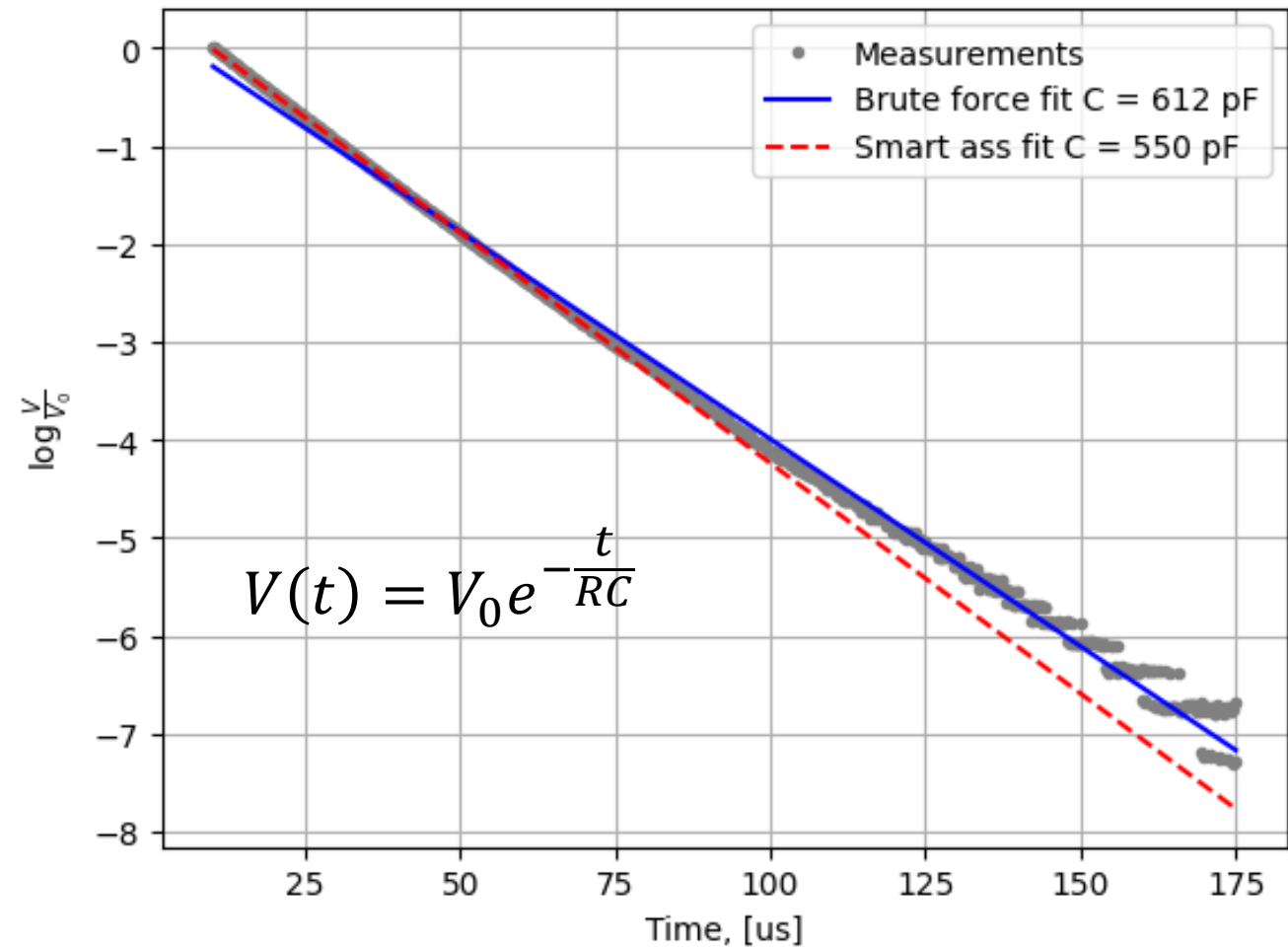
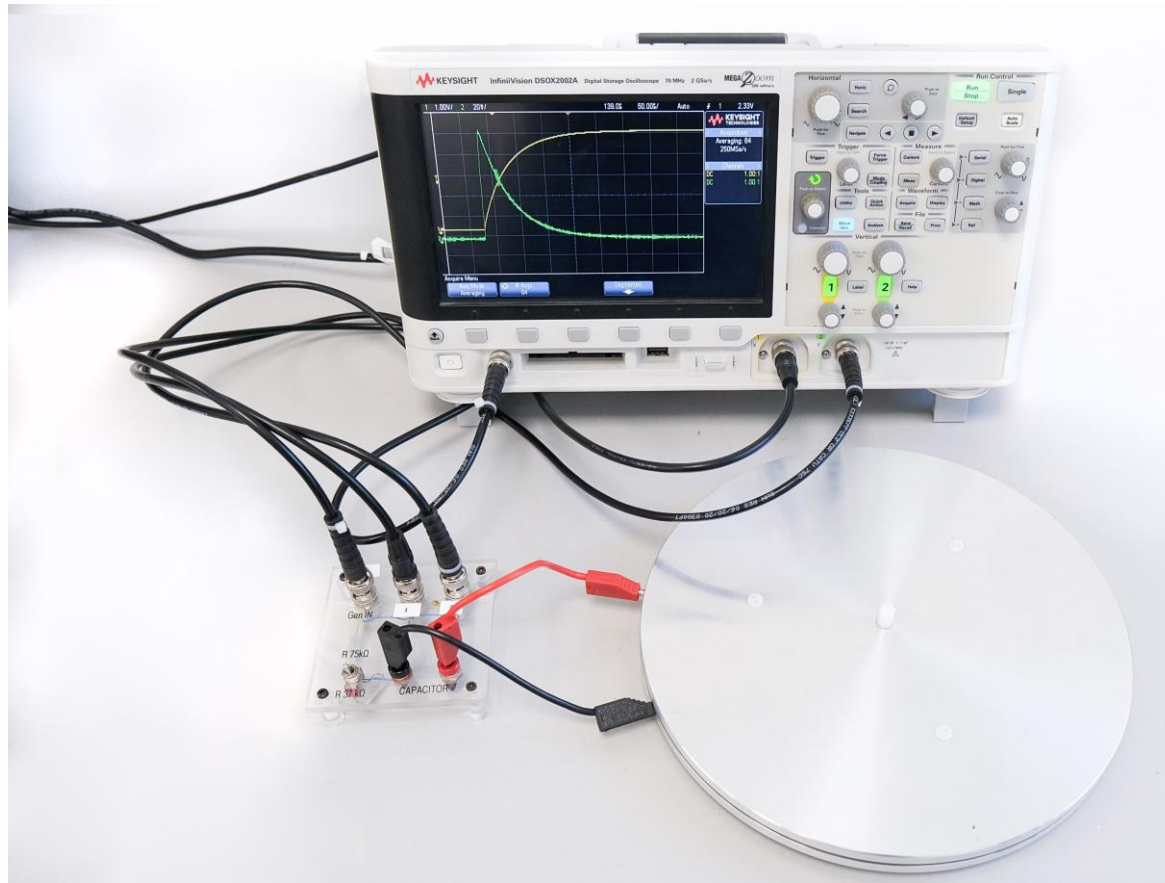
Slope : 2.7 ± 0.1

# בליעת קרינת $\gamma$ בעופרת



בכל מדידה שגיאה 3%

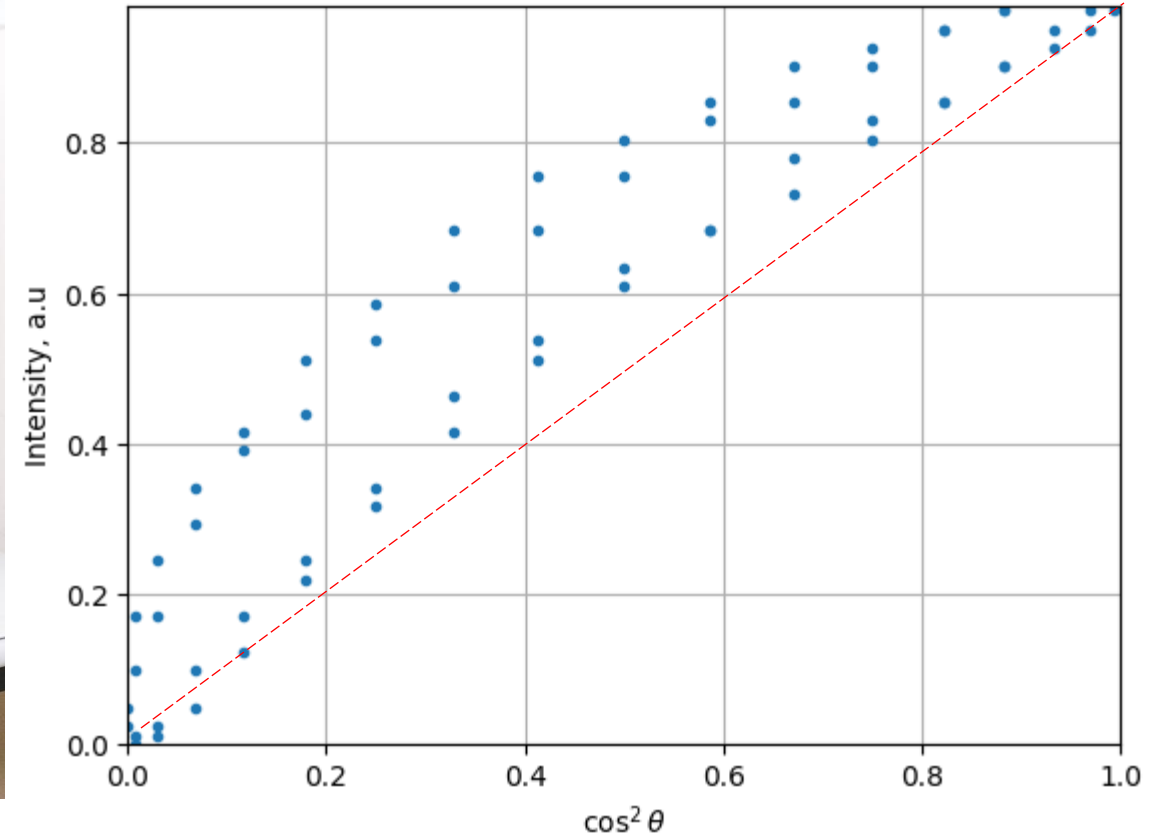
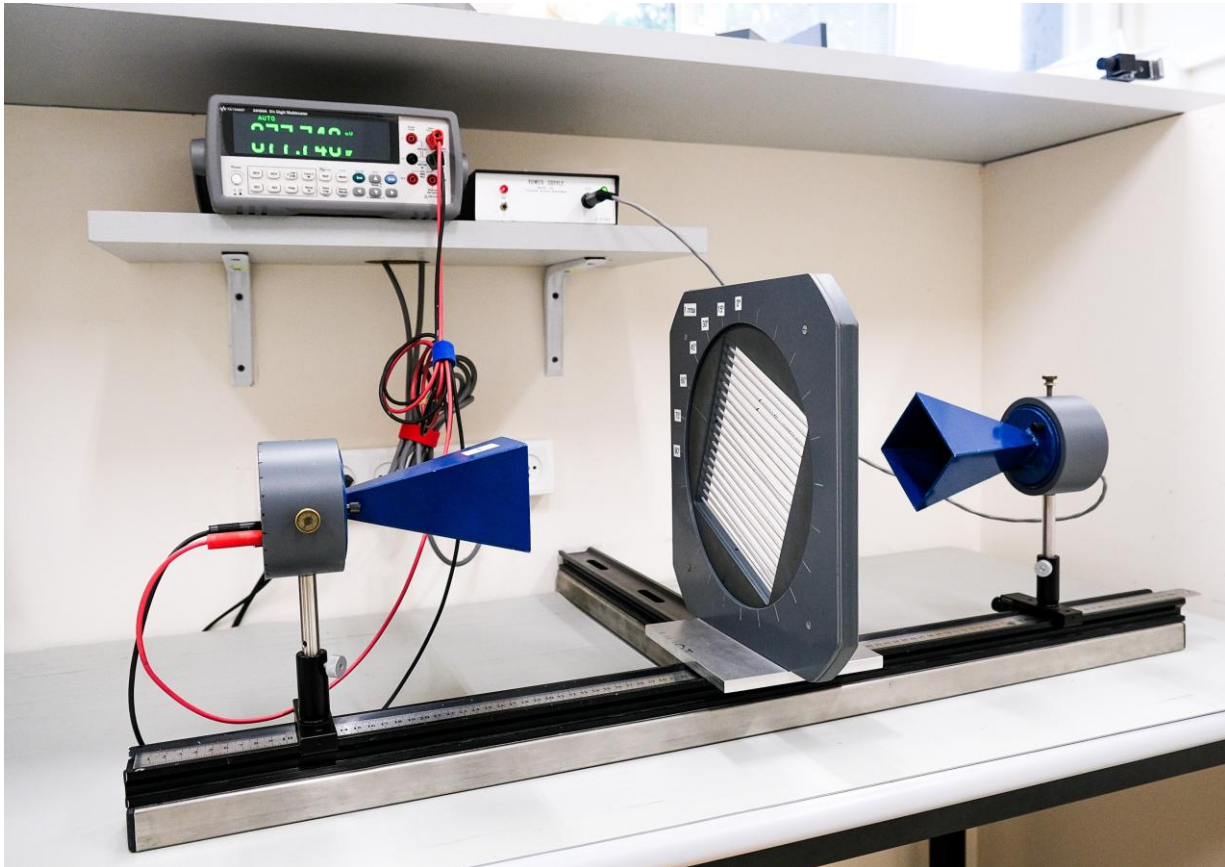
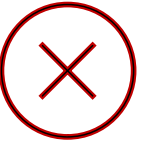
# פריקה של קבל לוחות





# קיטוב של גלי מיקרו

$$I = I_0 \cos^2 \theta$$



# התאמה לא-לינארית עם 4 פרמטרים

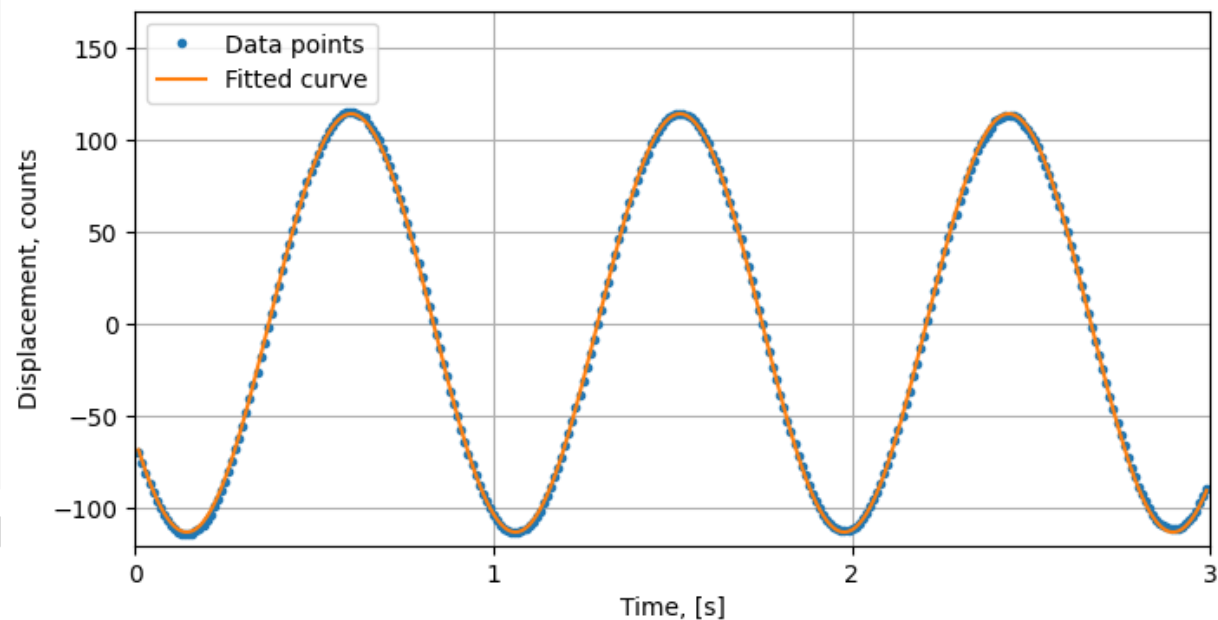
$$x = A \sin(\omega t + \varphi) + C$$

```
from scipy.optimize import curve_fit

def harmonic(x, amp, omega, phi, offset):
    return amp * np.sin(omega * x + phi) + offset

initial = [100, 6.6, 0, 0]
parameters, _ = curve_fit(harmonic, t, x, p0=initial)
x_expected = harmonic(t, *parameters)
print(parameters)
```

[-113.46582772 6.84902342 0.58338653 0.71716635]



לפעמים לכדאי להגדיר פרמטרים התחלתיים

# התאמה לא-לינארית עם 5 פרמטרים

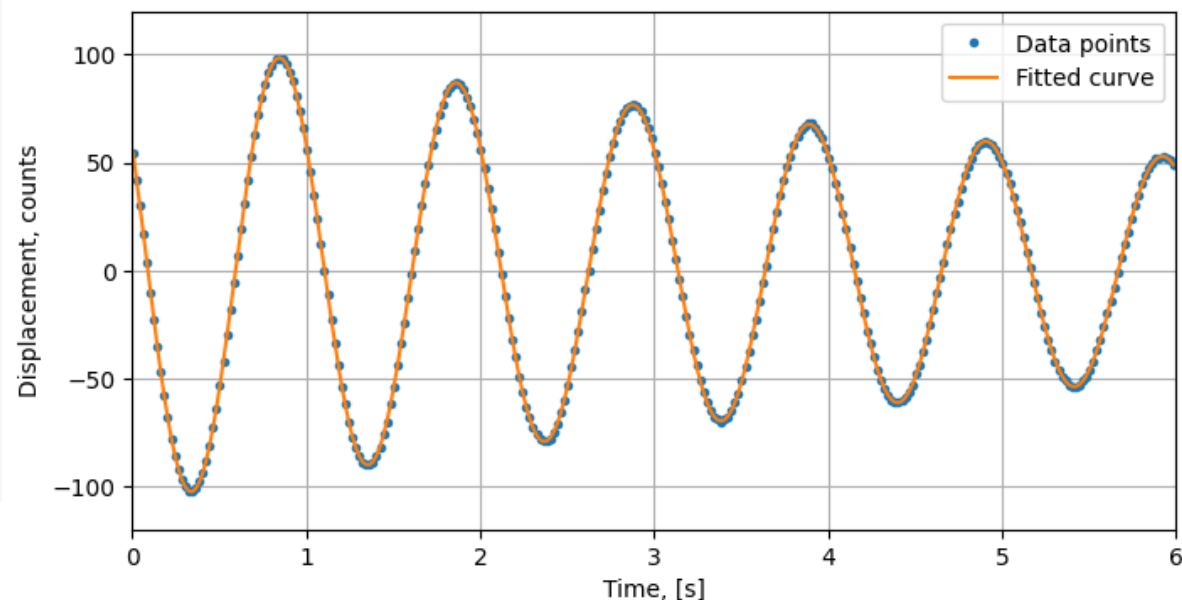
$$x = A e^{-t/2\tau} \sin(\omega t + \varphi) + C$$

```
from scipy.optimize import curve_fit

def harmonic(x, amp, tau, omega, phi, offset):
    return amp * np.exp(-t/2/tau)*np.sin(omega * x + phi) + offset

initial = [100, 20, 6.0, 0, 0]
bounds = ([ 75, 0, 5.0, -np.pi, -20],
          [125, 50, 7.0, np.pi, 20] )
parameters, _ = curve_fit(harmonic, t, x, p0=initial, bounds=bounds)
x_expected = harmonic(t, *parameters)
print(parameters)

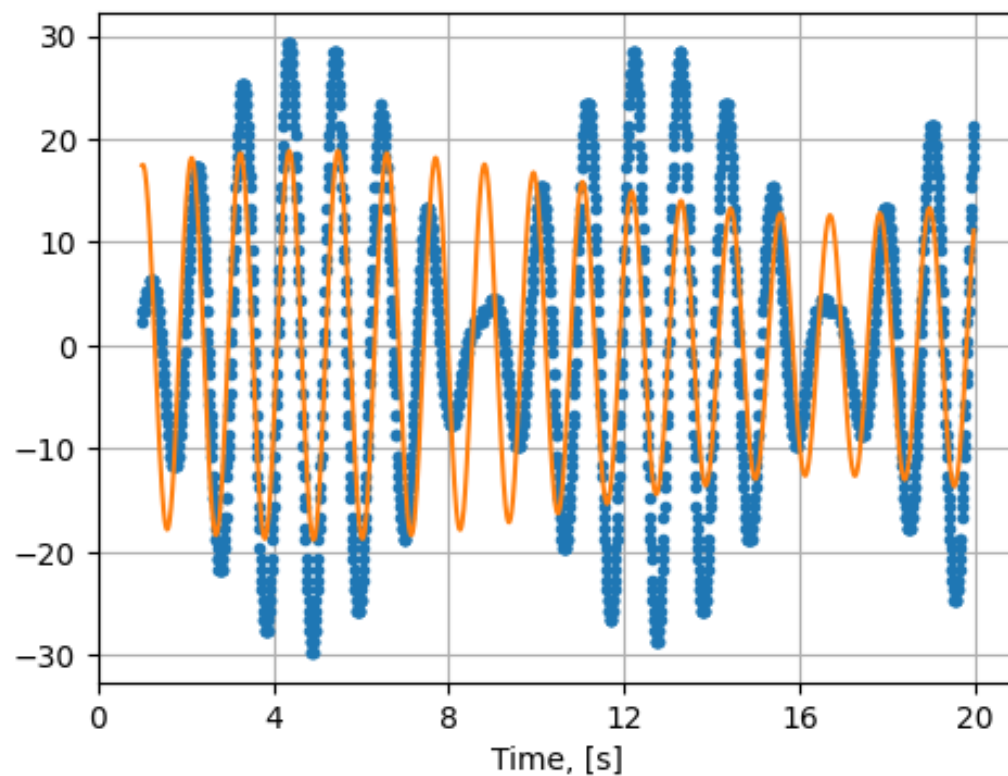
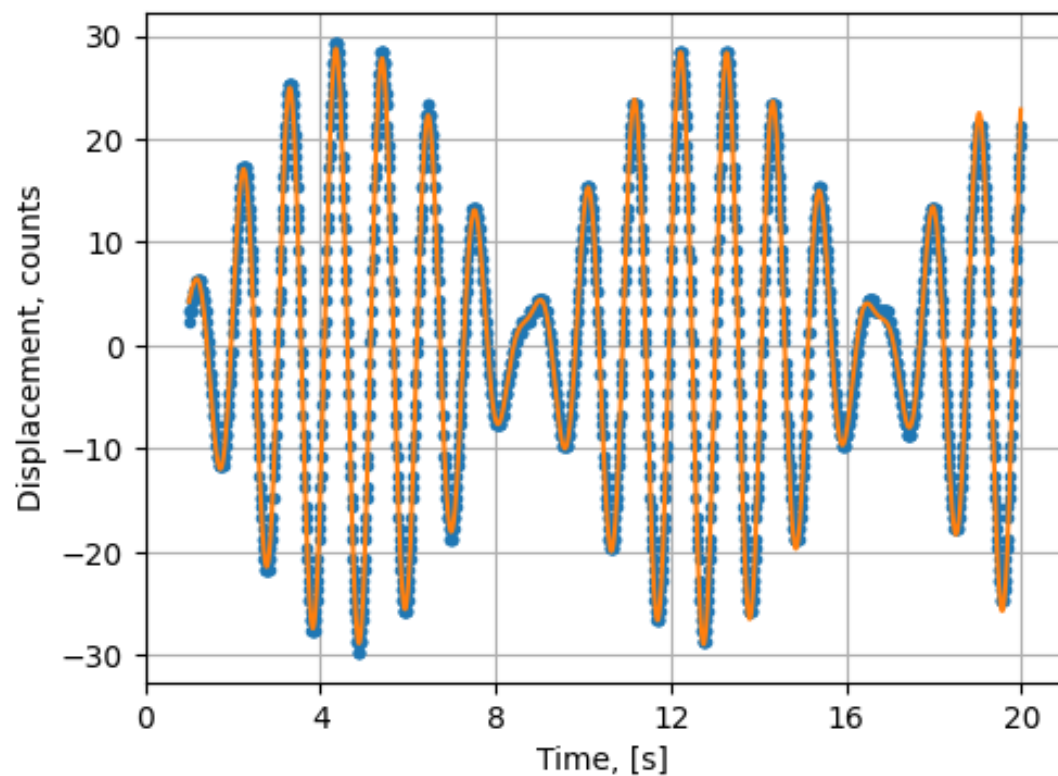
[108.20529269  4.01769347  6.18812637  2.59491801  1.17950182]
```



צריך להגדיר פרמטרים התחלתיים והגבלות למרחב פרמטרים

# התאמה לא-לינארית עם 6 פרמטרים

$$x = A \sin(\omega_0 t + \varphi) + B \sin(\omega_{dr} t + \psi)$$



# ספריה שיודעת לעשות Global optimization

```
from lmfit import Parameters, minimize

def model(params, x):
    amp1 = params['amp1']
    phase1 = params['phase1']
    omega1 = params['omega1']
    amp2 = params['amp2']
    phase2 = params['phase2']
    omega2 = params['omega2']
    return amp1 * np.sin(x*omega1 + phase1) + amp2 * np.sin(x*omega2 + phase2)

def residual(params, x, y, uncertainty):
    return (model(params, x) - y) / uncertainty

params = Parameters()
params.add('amp1', value=1, min=0, max=20)
params.add('phase1', value=0.0, min=-np.pi, max=np.pi)
params.add('omega1', value=1, min=1, max=10)
params.add('amp2', value=2, min=0, max=20)
params.add('phase2', value=0.0, min=-np.pi, max=np.pi)
params.add('omega2', value=1, min=1, max=10)

fit = minimize(residual, params, args=(x, y, uncertainty), method='dual_annealing')
y_expected = model(fit.params, x)
```

